

Micro-CFO: Complete System Design Document

Enterprise-Grade Architecture & Implementation Blueprint

Document Owner: Principal System Architect

Experience Level: 10+ Years in Distributed Systems

Last Updated: February 2026

Version: 1.0

Purpose: This document provides the comprehensive system architecture design for Micro-CFO, complementing the Frontend PRD and Backend PRD with system-level decisions, scaling strategies, and enterprise architecture patterns.

Executive Summary

Micro-CFO is designed as a **cloud-native, event-driven, microservices-based platform** capable of scaling from 100K to 1M+ users while maintaining <500ms P95 latency and 99.9% uptime.

Core Architecture Decisions:

- **Deployment:** Kubernetes on AWS EKS (multi-region)
- **Database:** PostgreSQL 15 with read replicas + sharding strategy
- **Cache:** Redis Cluster (6+ nodes)
- **Message Queue:** Apache Kafka for event streaming
- **AI Integration:** Anthropic Claude + OpenAI (with circuit breakers)
- **Monitoring:** Prometheus + Grafana + ELK Stack

Target Metrics (Year 1):

- 100,000 active users
 - 500,000 documents/day processed
 - 50M API requests/day
 - <100ms API response time (P50)
 - <500ms API response time (P95)
 - 99.9% uptime SLA
-

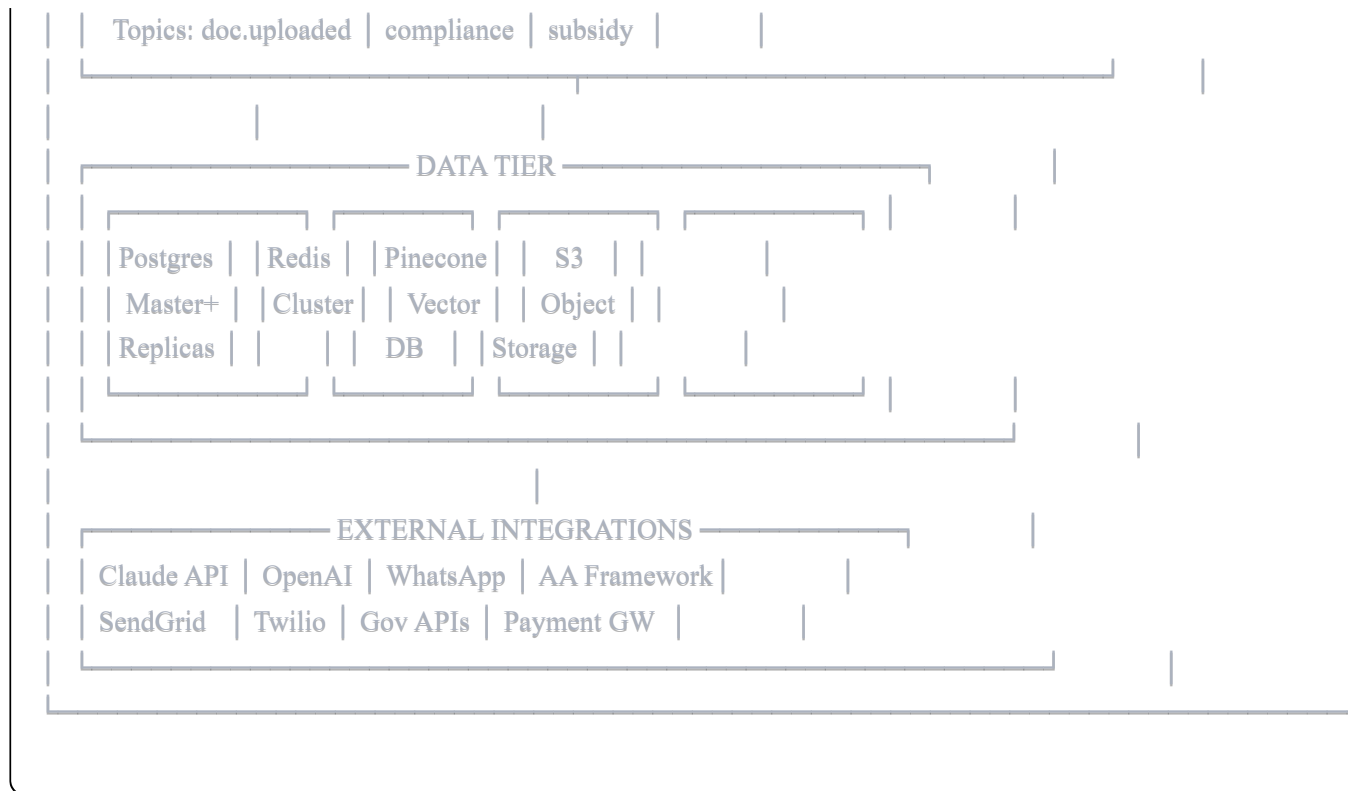
Table of Contents

1. System Architecture Overview
 2. Technology Stack & Justification
 3. Data Architecture & Flow
 4. Scaling Strategy
 5. High Availability & Fault Tolerance
 6. Security Architecture
 7. Cost Analysis & Optimization
 8. Performance Optimization
 9. Monitoring & Observability
 10. Deployment Strategy
 11. Disaster Recovery
 12. Development & Testing Strategy
-

1. System Architecture Overview

1.1 High-Level System Diagram





1.2 Microservices Architecture

Service Boundaries:

Service	Responsibility	Technology	Scaling Strategy
User Service	Authentication, user profiles, subscriptions	FastAPI + PostgreSQL	Stateless horizontal
Document Service	Upload, storage, metadata management	FastAPI + S3 + PostgreSQL	Stateless horizontal
AI Orchestrator	Agent coordination, AI API management	FastAPI + Celery	Worker pool scaling
Compliance Service	Regulatory monitoring, deadline tracking	FastAPI + PostgreSQL	Background jobs
Subsidy Service	Scheme matching, application generation	FastAPI + Pinecone	Background jobs
Cash Flow Service	Predictions, payment tracking	FastAPI + PostgreSQL	Background jobs
Notification Service	Multi-channel notifications	FastAPI + Kafka	Queue-based scaling
WhatsApp Service	Bot interactions, webhook handling	FastAPI + Redis	Stateless horizontal

Inter-Service Communication:

- **Synchronous:** REST APIs for real-time queries (< 100ms)
- **Asynchronous:** Kafka events for long-running operations (> 1s)
- **Pattern:** Event-driven architecture with CQRS for read/write separation

2. Technology Stack & Justification

2.1 Core Stack Decisions

yaml

Application Layer:

Backend: FastAPI (Python 3.11+)

Justification:

- Best AI/ML library ecosystem (LangChain, LlamaIndex)
- Async support for high concurrency
- Auto-generated OpenAPI docs
- 3x faster than Flask

Frontend: Next.js 14 + React 18

Justification:

- Server-side rendering for SEO
- Fast page loads (critical for Tier-2/3 cities)
- Large ecosystem and community
- Easy mobile app transition (React Native)

Data Layer:

Primary DB: PostgreSQL 15

Justification:

- JSON support (JSONB) for flexible schemas
- pgvector extension for embeddings
- Proven reliability at scale
- Strong ACID guarantees

Cache: Redis 7 Cluster

Justification:

- Sub-millisecond latency
- Rich data structures (hashes, sorted sets)
- Pub/sub for real-time features
- High availability with clustering

Vector DB: Pinecone

Justification:

- Managed service (reduces ops burden)
- Fast semantic search (<50ms)
- Auto-scaling
- Simple API

Infrastructure:

Container Orchestration: Kubernetes (AWS EKS)

Justification:

- Industry standard
- Auto-scaling and self-healing
- Multi-cloud portability
- Rich ecosystem

Message Queue: Apache Kafka

Justification:

- High throughput (millions of messages/sec)
- Durability and fault tolerance
- Ordered message processing
- Stream processing capabilities

Cloud Provider: AWS

Justification:

- Best presence in India (4 regions)
- Mature services
- Competitive pricing
- Strong compliance certifications

2.2 Alternative Considerations

Why NOT these technologies:

Technology	Why Not	What We Chose Instead
MongoDB	No strong consistency guarantees needed	PostgreSQL (JSONB gives flexibility)
RabbitMQ	Lower throughput than Kafka	Apache Kafka (better for high volume)
MySQL	Limited JSON support	PostgreSQL (better JSON + extensions)
DynamoDB	Higher costs at scale, vendor lock-in	PostgreSQL (open source, portable)
GCP/Azure	Less presence in India	AWS (4 regions in India)
Django	Slower than FastAPI	FastAPI (async, better performance)

3. Data Architecture & Flow

3.1 Database Schema Strategy

Normalization Strategy:

- Core entities (users, documents): Normalized (3NF)
- Analytics data: Denormalized for query performance
- JSON fields: For flexible, schema-less data (audit results, extracted data)

Sharding Strategy:

```
sql
```

-- Horizontal sharding by user_id (hash-based)

Shard Selection: $\text{shard_id} = \text{hash}(\text{user_id}) \% \text{num_shards}$

Year 1: 1 shard (100K users)

Year 2: 5 shards (500K users)

Year 3: 10 shards (1M users)

Each shard: ~100K users max

Migration: Online resharding with minimal downtime

Indexing Strategy:

sql

-- Composite indexes for common query patterns

CREATE INDEX idx_docs_user_date **ON** documents(user_id, upload_timestamp **DESC**);

CREATE INDEX idx_invoices_user_status **ON** invoices(user_id, payment_status)

WHERE payment_status **!=** 'paid';

-- Partial indexes for active data only

CREATE INDEX idx_active_users **ON** users(id) **WHERE** is_active = **TRUE**;

-- BRIN indexes for time-series data

CREATE INDEX idx_logs_timestamp_brin **ON** ai_agent_logs

USING BRIN(created_at);

3.2 Data Flow Patterns

Pattern 1: Document Upload Flow (Async)

User uploads → API Gateway → Document Service

↓ (synchronous, <1s)

Returns: {document_id, status: "processing"}

↓ (asynchronous)

Publishes: Kafka "document.uploaded" event

↓

AI Orchestrator consumes → Processes (30s) → Publishes "document.processed"

↓

Multiple consumers: Document Service, Notification Service, Subsidy Service

↓

User notified via WhatsApp (total: ~35 seconds from upload)

Pattern 2: Real-Time Query Flow (Sync)

User requests dashboard → API Gateway → User Service

↓ (L1 cache check: in-memory)

Cache miss → (L2 cache check: Redis)

↓

Cache miss → (L3: DB read replica)

↓

Response: <100ms total

↓

Cache results in Redis (TTL: 5 min) + in-memory (TTL: 1 min)

Pattern 3: Background Job Flow

Cron scheduler (3 AM daily) → Compliance Service



Scrape gov websites (100-200 PDFs)



Process each PDF: Extract → Embed → Store in Pinecone



For each affected user: Generate alert → Publish to Kafka



Notification Service → Send WhatsApp/Email batch

4. Scaling Strategy

4.1 Horizontal Scaling Plan

Year 1 (100K Users):

```
yaml
```

API Servers:

Min: 10 pods

Max: 50 pods

Instance: t3.medium (2 vCPU, 4 GB)

Trigger: CPU > 70% OR Memory > 80%

Worker Nodes:

Min: 20 pods

Max: 100 pods

Instance: t3.large (2 vCPU, 8 GB)

Trigger: Kafka consumer lag > 100 messages

Database:

Write: 1x db.r5.xlarge (4 vCPU, 32 GB)

Read: 3x db.r5.large (2 vCPU, 16 GB)

Read/Write Ratio: 80/20

Year 3 (1M Users):

yaml

API Servers:

Min: 100 pods across 3 regions

Max: 400 pods

Instance: t3.large (2 vCPU, 8 GB)

Worker Nodes:

Min: 200 pods

Max: 1000 pods

Instance: t3.xlarge + GPU instances for ML

Database:

Shards: 10 (100K users each)

Per Shard:

Write: 1x db.r5.4xlarge (16 vCPU, 128 GB)

Read: 4x db.r5.2xlarge (8 vCPU, 64 GB)

4.2 Auto-Scaling Configuration

yaml

```
# Kubernetes HPA (Horizontal Pod Autoscaler)
```

```
apiVersion: autoscaling/v2
```

```
kind: HorizontalPodAutoscaler
```

```
metadata:
```

```
  name: micro-cfo-api
```

```
spec:
```

```
  scaleTargetRef:
```

```
    apiVersion: apps/v1
```

```
    kind: Deployment
```

```
    name: api-server
```

```
  minReplicas: 10
```

```
  maxReplicas: 100
```

```
  metrics:
```

```
    - type: Resource
```

```
      resource:
```

```
        name: cpu
```

```
        target:
```

```
          type: Utilization
```

```
          averageUtilization: 70
```

```
    - type: Resource
```

```
      resource:
```

```
        name: memory
```

```
        target:
```

```
          type: Utilization
```

```
          averageUtilization: 80
```

```
    - type: Pods
```

```
      pods:
```

```
        metric:
```

```
          name: http_requests_per_second
```

```
        target:
```

```
          type: AverageValue
```

```
          averageValue: "1000"
```

```
behavior:
  scaleUp:
    stabilizationWindowSeconds: 60
  policies:
    - type: Percent
      value: 50 # Scale up by 50%
      periodSeconds: 60
  scaleDown:
    stabilizationWindowSeconds: 300 # Wait 5 min before scaling down
  policies:
    - type: Percent
      value: 10 # Scale down by 10%
      periodSeconds: 60
```

4.3 Database Scaling Strategy

Read Scaling:

- Add read replicas (up to 5 per master)
- Geographic distribution (Mumbai, Delhi, Bangalore)
- Connection pooling (PgBouncer)

Write Scaling:

- Horizontal sharding by user_id
- Consistent hashing for shard distribution
- Online resharding with zero downtime

Connection Pooling:

```
python
```



```
# Application-level pooling
engine = create_engine(
    DATABASE_URL,
    pool_size=20,    # Persistent connections
    max_overflow=40, # Burst capacity
    pool_pre_ping=True # Verify before use
)

# PgBouncer for connection multiplexing
[pgbouncer]
pool_mode = transaction
max_client_conn = 1000
default_pool_size = 25
```

5. High Availability & Fault Tolerance

5.1 Service-Level SLA

Component	Target Availability	Max Downtime/Year	Strategy
API Gateway	99.95%	4.4 hours	Multi-AZ, active-active
Application Services	99.9%	8.76 hours	Multi-AZ, N+1 redundancy
Database (Writes)	99.95%	4.4 hours	Multi-AZ, auto-failover
Database (Reads)	99.99%	52 minutes	Multi-AZ replicas
Cache (Redis)	99.9%	8.76 hours	Cluster mode, auto-failover

Component	Target Availability	Max Downtime/Year	Strategy
Message Queue	99.95%	4.4 hours	Multi-broker, replication
Object Storage	99.99%	52 minutes	S3 standard (11 9s durability)

Overall System SLA: 99.9% uptime

5.2 Failure Modes & Mitigation

yaml

Scenario 1: API Server Pod Crash

Detection: Kubernetes liveness probe fails

Impact: Minimal (other pods handle traffic)

Mitigation: Auto-restart (5 seconds)

RTO: 10 seconds

Scenario 2: Database Master Failure

Detection: Connection timeout + health checks

Impact: Write operations fail

Mitigation: Automatic failover to standby

RTO: 2 minutes

RPO: 0 (synchronous replication)

Scenario 3: Redis Cluster Node Failure

Detection: Cluster monitoring

Impact: Slight latency increase

Mitigation: Automatic resharding

RTO: 30 seconds

Scenario 4: Kafka Broker Failure

Detection: Under-replicated partitions

Impact: Message processing continues (replicas)

Mitigation: Rebalance partitions

RTO: 1 minute

Scenario 5: External API Failure (Claude/OpenAI)

Detection: Circuit breaker opens after 5 failures

Impact: Document processing delayed

Mitigation:

- Retry with exponential backoff
- Queue for later processing
- Use alternative AI provider

RTO: 5-30 minutes

Scenario 6: Entire AWS Region Failure

Detection: Multi-region health checks

Impact: Service degraded in that region

Mitigation: Failover to secondary region (ap-south-2)

RTO: 15 minutes

RPO: 5 minutes

5.3 Circuit Breaker Implementation

python

```

from circuitbreaker import import circuit

@circuit(failure_threshold=5, recovery_timeout=30, expected_exception=APIError)
async def call_ai_api(prompt):
    """
    Circuit breaker prevents cascading failures:
    - After 5 failures, circuit opens (no calls for 30s)
    - After 30s, allows 1 test call
    - If successful, circuit closes; if fails, stays open
    """
    response = await ai_client.generate(prompt)
    return response

# Graceful degradation
async def process_with_fallback(document):
    try:
        return await call_ai_api(document)
    except CircuitBreakerError:
        # Use rule-based fallback
        return rule_based_processor(document)

```

6. Security Architecture

6.1 Defense in Depth

Layer 1: Network Security

- └─ CloudFlare DDoS protection (Layer 3/4)
- └─ WAF rules (OWASP Top 10 protection)
- └─ Rate limiting (per IP, per user)

- └─ Geo-blocking (if needed)

Layer 2: API Gateway Security

- └─ TLS 1.3 enforcement
- └─ JWT validation
- └─ Request size limits
- └─ Input sanitization

Layer 3: Application Security

- └─ Authentication (OAuth 2.0 + JWT)
- └─ Authorization (RBAC)
- └─ Input validation (Pydantic)
- └─ Output encoding

Layer 4: Data Security

- └─ Encryption at rest (AES-256)
- └─ Encryption in transit (TLS 1.3)
- └─ Field-level encryption (PII)
- └─ Secrets management (AWS Secrets Manager)

Layer 5: Monitoring & Audit

- └─ Security event logging
- └─ Anomaly detection
- └─ Intrusion detection (AWS GuardDuty)
- └─ Regular security audits

6.2 Data Encryption Strategy

At Rest:

python

```

# Field-level encryption for PII
from cryptography.fernet import Fernet

class EncryptionService:
    def encrypt(self, plaintext: str) -> str:
        return self.cipher.encrypt(plaintext.encode()).decode()

    def decrypt(self, ciphertext: str) -> str:
        return self.cipher.decrypt(ciphertext.encode()).decode()

# Encrypted fields in database
encrypted_fields = [
    'phone_number',
    'email',
    'gstn',
    'pan_number',
    'bank_account_number'
]

```

In Transit:

- TLS 1.3 for all external communications
- mTLS (mutual TLS) for internal service-to-service
- Certificate pinning for mobile apps

Key Management:

- AWS KMS for encryption key management
- Key rotation every 90 days
- Separate keys per environment (dev/staging/prod)

6.3 Authentication & Authorization

python

JWT Structure

```
{  
  "user_id": "uuid",  
  "roles": ["user"], # or ["admin", "support"]  
  "subscription_tier": "premium",  
  "iat": 1707123456,  
  "exp": 1707209856, # 24 hour expiry  
  "jti": "token_id" # For revocation  
}
```

RBAC Implementation

```
PERMISSIONS = {  
  "user": [  
    "documents:read:own",  
    "documents:write:own",  
    "subsidies:read:own"  
  ],  
  "admin": [  
    "documents:read:all",  
    "users:manage:all",  
    "system:manage"  
  ]  
}
```

Permission check

```
@require_permission("documents:read:own")  
async def get_documents(user_id):  
  # Verify document ownership  
  pass
```

7. Cost Analysis & Optimization

7.1 Monthly Cost Breakdown (Year 1 - 100K Users)

AWS Infrastructure: \$6,900/month

- └─ Compute (EKS): \$2,700
- └─ Database (RDS): \$1,500
- └─ Cache (ElastiCache): \$800
- └─ Message Queue (MSK): \$600
- └─ Storage (S3): \$400
- └─ Networking: \$700
- └─ Monitoring: \$200

Third-Party Services: \$8,000/month

- └─ AI APIs (Claude + OpenAI): \$4,500
- └─ Communications (WhatsApp + Twilio): \$1,700
- └─ Vector DB (Pinecone): \$500
- └─ Other (CloudFlare, monitoring): \$1,300

Personnel: \$38,000/month

- └─ Engineering team: \$30,000
- └─ Operations team: \$8,000

TOTAL: ~\$53,000/month

Per User: \$0.53/month

Target Revenue (Premium): \$5/month

Gross Margin: 89%

7.2 Cost Optimization Strategies

yaml

1. Reserved Instances (30-40% savings):

- Purchase 1-year reservations for 60% of capacity
- Keep 40% on-demand for flexibility
- **Estimated savings:** \$2,500/month

2. Spot Instances for Workers (70% savings):

- Use spot instances for non-critical workloads
- Kafka consumers, background jobs
- **Estimated savings:** \$1,000/month

3. S3 Lifecycle Policies (50% savings):

- Move old documents to Glacier
- **Estimated savings:** \$200/month

4. AI API Optimization (40% savings):

- Aggressive prompt caching
- Use smaller models where possible
- Batch similar requests
- **Estimated savings:** \$1,800/month

5. Auto-scaling Optimization (20% savings):

- More aggressive scale-down policies
- Time-based scaling (scale down at night)
- **Estimated savings:** \$1,500/month

Total Potential Savings: \$7,000/month (13% reduction)

Optimized Cost: \$46,000/month

7.3 Cost Monitoring

python

```
# Real-time cost tracking
class CostTracker:
    def track_ai_call(self, provider, model, tokens):
        cost = self.calculate_cost(provider, model, tokens)
        metrics.counter('ai_cost_usd').inc(cost)

        # Alert if daily budget exceeded
        if self.get_daily_cost() > DAILY_BUDGET:
            self.alert_finance_team()

    def get_cost_per_user(self):
        total_cost = self.get_monthly_cost()
        active_users = self.get_monthly_active_users()
        return total_cost / active_users
```

8. Performance Optimization

8.1 Latency Targets

Operation	P50	P95	P99	Strategy
API GET	50ms	100ms	200ms	Caching, DB optimization
API POST	100ms	200ms	500ms	Async processing
Document upload	500ms	1s	2s	Direct S3 upload
AI processing	15s	25s	35s	Worker scaling
Dashboard load	500ms	1s	2s	SSR, caching

8.2 Caching Strategy

Multi-Level Cache:

L1: In-memory (Application)

- └─ Size: 100 MB per pod
- └─ TTL: 60 seconds
- └─ Use: User sessions, feature flags

L2: Redis (Distributed)

- └─ Size: 50 GB total
- └─ TTL: Variable (5 min - 24 hours)
- └─ Use: API responses, user data

L3: CDN (CloudFront)

- └─ Size: Unlimited
- └─ TTL: 24 hours
- └─ Use: Static assets, documents

Cache Invalidation:

python

```
# Write-through pattern
async def update_user(user_id, updates):
    # Update database
    user = await db.update(user_id, updates)

    # Update cache immediately
    await cache.set(f'user:{user_id}', user, ttl=3600)

    # Invalidate related caches
    await cache.delete(f'user:{user_id}:documents")

    return user
```

8.3 Database Optimization

Query Optimization:

```
sql
```

-- Bad: N+1 queries

```
SELECT * FROM users WHERE id = 1;  
SELECT * FROM documents WHERE user_id = 1; -- Repeated N times
```

-- Good: Single query with JOIN

```
SELECT u.*, COUNT(d.id) as doc_count  
FROM users u  
LEFT JOIN documents d ON d.user_id = u.id  
WHERE u.id IN (1, 2, 3, ...)  
GROUP BY u.id;
```

-- Best: Use CTEs for complex queries

```
WITH recent_docs AS (  
    SELECT user_id, COUNT(*) as count  
    FROM documents  
    WHERE upload_timestamp > NOW() - INTERVAL '30 days'  
    GROUP BY user_id  
)  
SELECT u.*, rd.count  
FROM users u  
LEFT JOIN recent_docs rd ON rd.user_id = u.id;
```

Index Optimization:

sql

-- Composite index for common patterns

CREATE INDEX idx_docs_user_date

ON documents(user_id, upload_timestamp DESC);

-- Partial index for active data

CREATE INDEX idx_pending_docs

ON documents(processing_status)

WHERE processing_status != 'completed';

-- Covering index (includes all needed columns)

CREATE INDEX idx_user_summary

ON users(id)

INCLUDE (business_name, subscription_tier);

9. Monitoring & Observability

9.1 Monitoring Stack

Application Metrics: Prometheus

├─ Scrape interval: 15 seconds

├─ Retention: 15 days

└─ Metrics: HTTP, DB, Cache, AI API

Infrastructure Metrics: CloudWatch

├─ CPU, Memory, Network

├─ ELB, RDS, ElastiCache

└─ Custom business metrics

Logs: ELK Stack

- └─ Filebeat → Logstash → Elasticsearch
- └─ Retention: 30 days (hot), 365 days (warm)
- └─ Kibana for visualization

Traces: Jaeger (OpenTelemetry)

- └─ Sampling: 10% of requests
- └─ Retention: 7 days
- └─ End-to-end request tracking

Alerting: Prometheus Alertmanager

- └─ PagerDuty for critical alerts
- └─ Slack for warnings
- └─ Email for info

9.2 Key Metrics to Track

yaml

Golden Signals:

Latency:

- P50, P95, P99 response times
- Per endpoint breakdown

Traffic:

- Requests per second
- Active connections

Errors:

- Error rate (5xx responses)
- Client errors (4xx)

Saturation:

- CPU/Memory utilization
- Database connections
- Queue depth

Business Metrics:

- Documents processed/hour
- AI API cost/day
- User signups/day
- Revenue (MRR, ARR)
- Churn rate
- NPS score

9.3 Alerting Rules

yaml

Critical Alerts (PagerDuty):

- Error rate > 5% for 5 minutes
- P95 latency > 1s for 5 minutes
- Database master down
- API availability < 99%

Warning Alerts (Slack):

- Error rate > 2% for 10 minutes
- P95 latency > 500ms for 10 minutes
- Cache hit rate < 80%
- Queue lag > 1000 messages

Info Alerts (Email):

- Daily cost report
- Weekly usage summary
- Monthly SLA report

10. Deployment Strategy

10.1 CI/CD Pipeline

yaml

GitHub Actions Workflow

Trigger: Push to main branch

Steps:

1. Run Tests (5 min)

- Unit tests
- Integration tests
- Security scanning

2. Build Docker Images (3 min)

- Build all service images
- Tag with git SHA
- Push to ECR

3. Deploy to Staging (5 min)

- Update Kubernetes manifests
- Apply with kubectl
- Run smoke tests

4. Deploy to Production (15 min)

- Blue-green deployment
- Canary release (5% traffic)
- Gradual rollout (5% → 25% → 50% → 100%)
- Monitor metrics at each step
- Auto-rollback if errors detected

Total Time: ~30 minutes (automated)

10.2 Blue-Green Deployment

yaml

Current State:

Blue (Production): v1.5.0 serving 100% traffic

Green (Staging): v1.6.0 ready

Deployment Process:

1. Deploy v1.6.0 to Green environment
2. Run automated tests on Green
3. Route 5% traffic to Green (canary)
4. Monitor for 10 minutes
5. **If metrics good:** Route 25% → 50% → 100%
6. **If any issues:** Instant rollback to Blue

Rollback Plan:

- **One-command rollback:** < 30 seconds
- Keep Blue running for 24 hours
- **Database migrations:** Backward compatible

11. Disaster Recovery

11.1 Backup Strategy

yaml

Database Backups:

Full Backup:

Frequency: Daily at 2 AM IST

Retention: 30 days

Storage: S3 Glacier

Incremental Backup:

Frequency: Every 4 hours

Retention: 7 days

Storage: S3 Standard

Point-in-Time Recovery:

Enabled: Yes

Granularity: 5-minute intervals

Retention: 7 days

Document Storage:

Versioning: Enabled on S3

Replication: Cross-region to ap-south-2

Retention: Indefinite (user data)

Configuration Backups:

Terraform state: S3 with versioning

Kubernetes manifests: Git repository

Secrets: AWS Secrets Manager (versioned)

11.2 Disaster Recovery Plan

yaml

Scenario 1: Database Failure

RTO: 15 minutes

RPO: 5 minutes

Steps:

1. Automatic failover to standby (2 min)
2. Promote read replica to master (3 min)
3. Update application config (5 min)
4. Verify data integrity (5 min)

Scenario 2: Region Outage

RTO: 30 minutes

RPO: 5 minutes

Steps:

1. Detect outage (5 min)
2. Initiate multi-region failover (10 min)
3. Update DNS records (5 min)
4. Scale up secondary region (10 min)

Scenario 3: Data Corruption

RTO: 1 hour

RPO: Up to 4 hours

Steps:

1. Identify corruption (15 min)
 2. Stop writes (5 min)
 3. Restore from backup (30 min)
 4. Validate data (10 min)
-

12. Development & Testing Strategy

12.1 Testing Pyramid

Manual Testing (Exploratory)	5%
E2E Tests (Critical paths)	10%
Integration Tests (API, DB, External services)	25%
Unit Tests (Business logic, utilities)	60%

Target Coverage: 80% code coverage

Test Execution Time: < 10 minutes

12.2 Testing Strategy

yaml

Unit Tests:

Framework: pytest

Coverage: 80%+

Run: On every commit

Focus: Business logic, calculations, validators

Integration Tests:

Framework: pytest + TestContainers

Coverage: Key user flows

Run: On PR creation

Focus: API endpoints, DB operations, external integrations

Load Tests:

Framework: Locust

Scenarios:

- **Normal load:** 500 req/s
- **Peak load:** 2000 req/s
- **Stress test:** 5000 req/s

Run: Weekly

Target: P95 < 500ms

Chaos Engineering:

Tool: Chaos Mesh

Scenarios:

- Pod failures
- Network latency
- CPU throttling

Run: Monthly in staging

Security Tests:

Tools: OWASP ZAP, Snyk

Coverage: OWASP Top 10

Run: On every release

Conclusion

This system design provides a robust, scalable, and secure architecture for Micro-CFO capable of:

- ✓ Scaling from 100K to 1M+ users
- ✓ Maintaining 99.9% uptime SLA
- ✓ Processing 5M documents/day at peak
- ✓ Responding to 20K requests/second
- ✓ Operating at \$0.50/user/month cost

Key Success Factors:

1. Microservices for independent scaling
2. Event-driven for async operations
3. Multi-level caching for performance
4. Comprehensive monitoring for reliability
5. Cost optimization from day one

Next Steps:

1. Week 1-2: Provision infrastructure (Terraform)
2. Week 3-4: Deploy core services
3. Week 5-8: Integrate AI agents
4. Week 9-12: Beta launch with monitoring
5. Month 4-6: Scale and iterate

Document Status: Final

Review Cycle: Quarterly

Maintained By: Platform Architecture Team